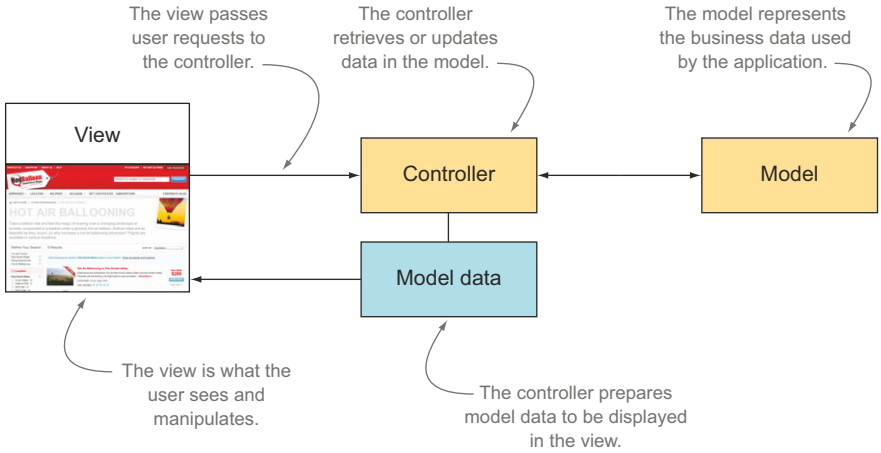


Figure 9.5 The MVC architecture pattern aims at cleanly separating application data from the way it's presented.

referred to as testing "under the skin" of the application. When you automate acceptance tests for a web application at the controller level, for example, you simulate requests arriving at a controller, and then check which page the controller displays and what data the controller places in the HTTP session or request scope.

In the Java world, Spring is an excellent and popular application development framework that provides first-class support for testing the controller layer. The web application module, Spring MVC, makes it easy to both write and test controllers using Java annotations to handle web requests.[1]

Suppose you're writing a Spring MVC application to monitor real-time flights for Flying High. You need a page that displays the current flight status for a particular flight. The corresponding acceptance criteria might look like this:

```
Feature: Displaying flight status

Scenario: Provide a positive visual queue for on-time flights
Given that flight FH-101 has no reported delays
When I check the flight status
Then I should see that it is on time
And I should see its scheduled arrival time
```

In a Spring MVC application, the view is typically a JSP page template that renders an HTML page containing data from the model. The controller receives queries from the user, prepares the model data to be displayed, and decides what page to render. For the flight status screen, the controller might receive the fight number as a parameter,

[1] For more information, see section 11.3.6, "Spring MVC Test Framework," of the Spring Framework Reference Documentation: http://docs.spring.io/spring/docs/3.2.x/spring-framework-reference/html/testing.html#spring-mvc-test-framework.